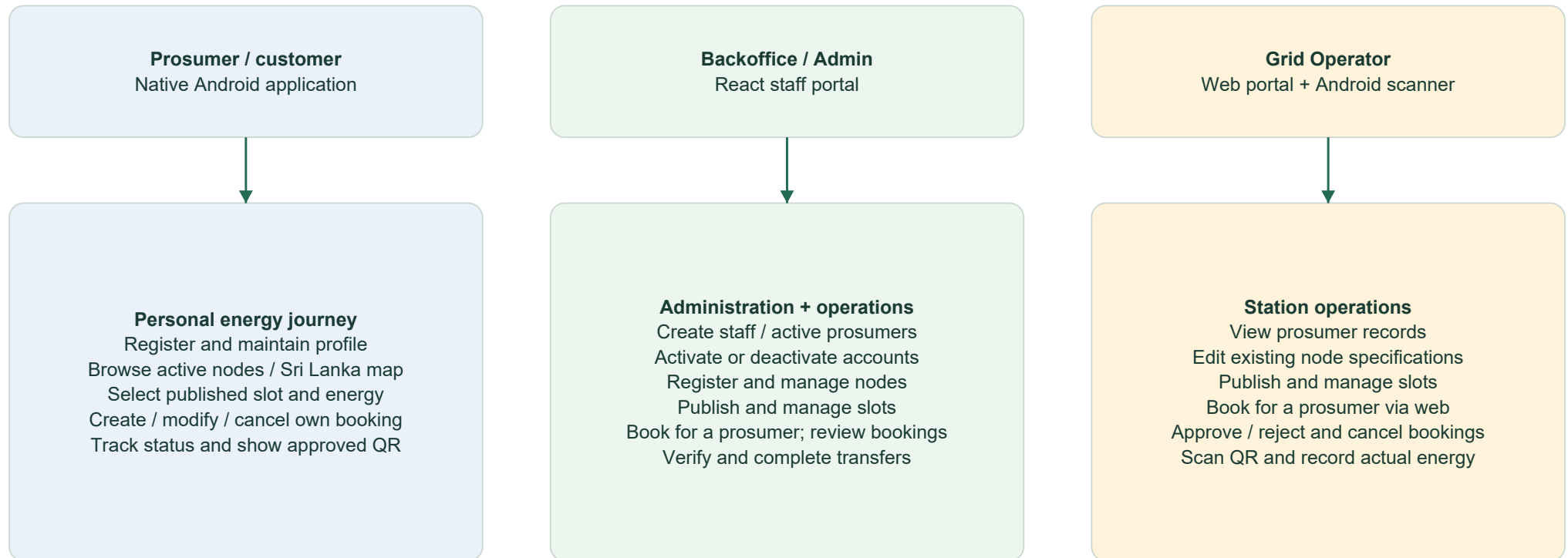


High-level design

03 / Responsibilities, data flows, rules and operational behavior



Goal: let active prosumers reserve published microgrid energy slots and let staff govern supply, review requests and verify completion. The central API owns all business decisions.

Read this guide for module boundaries and behavior. Read the separate LLD for classes, fields, endpoint contracts and transaction algorithms.

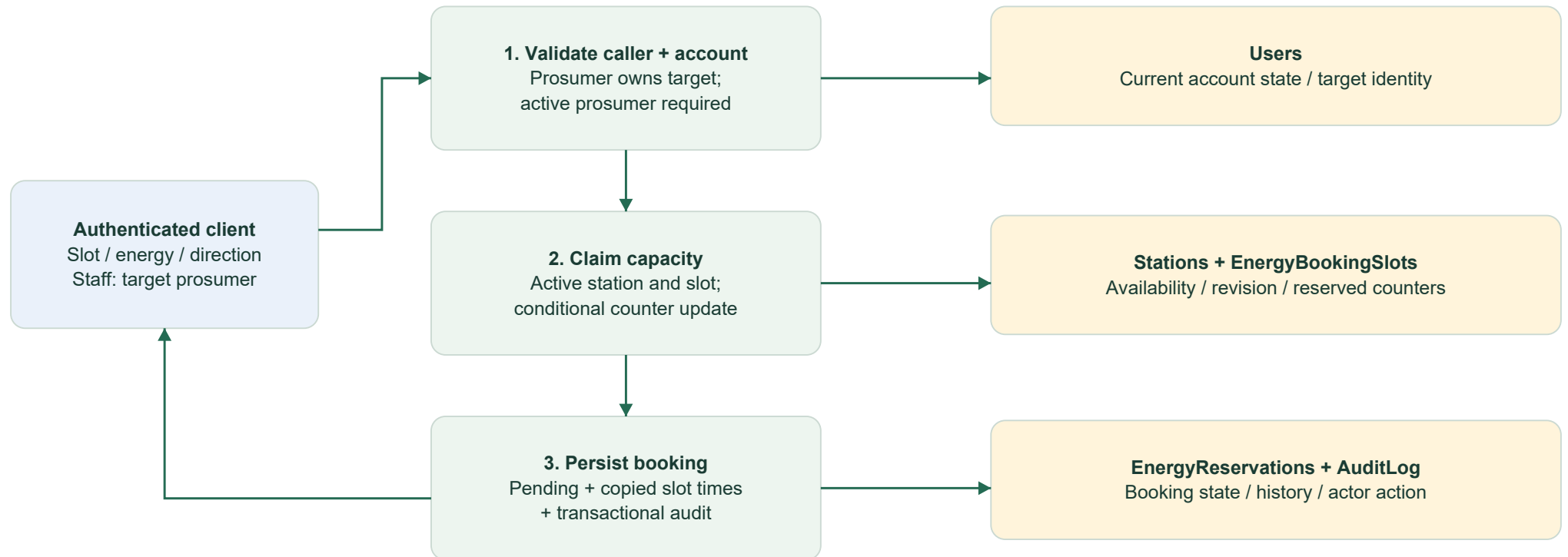
Subsystem decomposition

One API deployment with cohesive service modules and two clients.

Subsystem	Responsibility	Main data / interactions
Account and identity	NIC-based registration, login, staff creation, activation, profile edits and session revocation	Users; safe UserView; JWT validation queries current account state.
Grid supply	Node specifications, active state, future slots, overlap protection, node/battery constraints	SolarStationInfo + EnergyBookingSlots; stations feed both portal and mobile map.
Reservation management	Create, modify, cancel, staff review, summaries, search, paging and dashboard	EnergyReservations + held slot counters; ownership and time rules.
Transfer verification	Signed approved QR; live validation; one-time actual-energy completion	Current reservation nonce/status; scheduled window; completion actor/time/energy.
Background maintenance	Index/bootstrap startup and periodic expiry of abandoned reservations	Mongo indexes; optional first Backoffice; expire and release holds every minute.
Client presentation	Staff portal; native prosumer flow; operator scanning; full-island map; reference cache	REST only; browser sessionStorage; Android SQLite + Keystore.

Logical data flow

Reservation creation moves through validation, capacity claim and persistence in one transaction.



The arrows carry application data, not physical electricity. The station and slot records describe availability; they do not command hardware.

Supply setup and availability rules

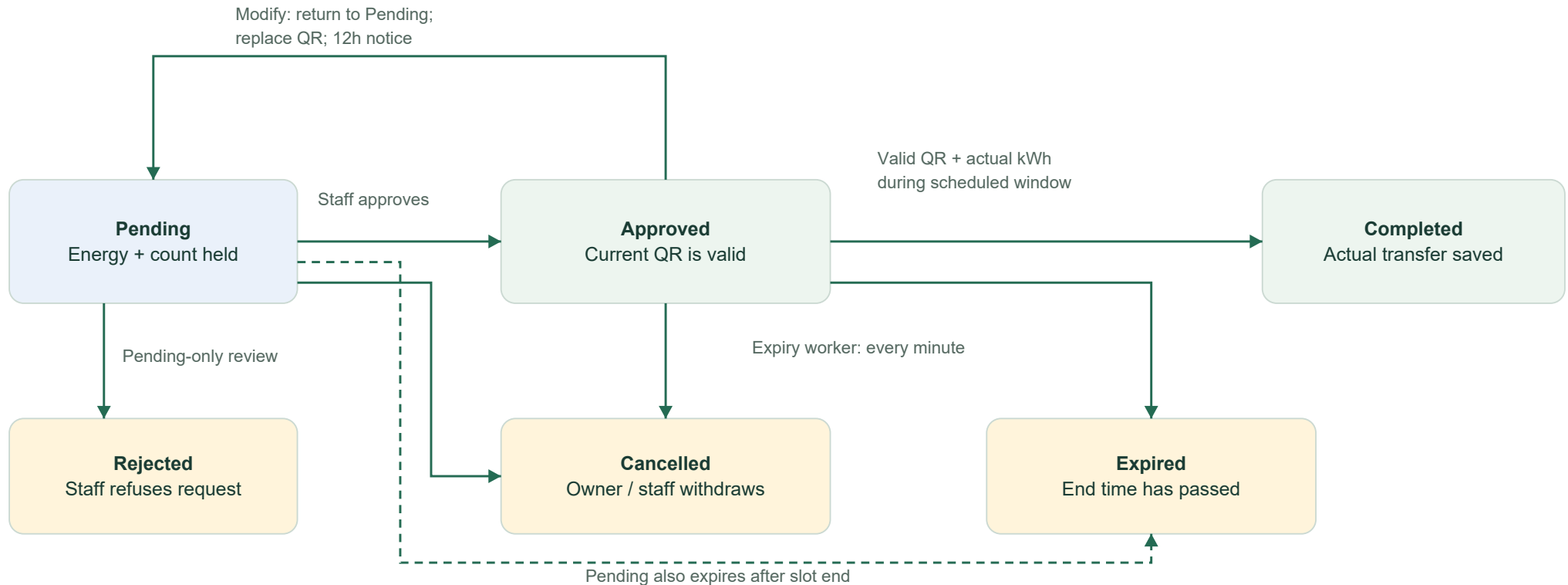
Configuration is protected against concurrent reservations.

Operation	Business constraint	Outcome
Register / edit node	Valid GPS; finite capacity; positive battery slots; reducing batteries must not conflict with active slot booking limits	Store station metadata. CapacityKw is generation power; it is not directly substituted for slot kWh.
Deactivate / archive node	Reject if any Pending or Approved reservation exists	Set Active=false; preserve station and booking history. Only Backoffice can perform lifecycle changes.
Publish slot	Active node; future start; end after start; positive energy; booking limit within node battery slots; no overlapping active slot at same node	Store a future interval and capacity. Touch station revision to serialize conflicting writes.
Edit / archive slot	Cannot edit or archive when ReservedCount > 0	Keep old booking contracts stable; archived slot keeps historical references.
List/map supply	Prosumers see active nodes and future active slots from active nodes	Mobile map calls unfiltered /stations; full island viewport. Listings are capped at 500 records.

Availability: free energy = CapacityKwh - ReservedKwh; free bookings = MaxBookings - ReservedCount. Both DropOff and Charging use the same reservation counters in the current implementation.

Reservation policy and state design

The server uses UTC instants; clients display local time.



New booking: $\text{now} < \text{start} \leq \text{now} + 7 \text{ days}$. Modify/cancel: $\text{start} \geq \text{now} + 12 \text{ hours}$. Modification must satisfy notice for both old and selected new slot. Review: only Pending and before start. Completion: $\text{start} \leq \text{now} \leq \text{end}$, Approved and valid QR.

Actual transfer energy is positive and at most reserved energy. Cancellation, rejection, expiry and completion release the full reservation hold; completion records actual energy separately. The slot counters model current holds, not cumulative physical throughput.

Security, consistency and failure behavior

These describe implemented controls; they are not performance or uptime guarantees.

Concern	Current design
Authorization / ownership	Role attributes guard staff-only actions. Prosumer reservation reads/writes must match ActorId. UsersController restricts operators to prosumer records.
Concurrent booking	Conditional slot update checks both count and energy; booking, counter and audit writes commit together. Mongo transaction retry handles conflicts. Station revision writes serialize supply/booking races.
Session revocation	Account activation/deactivation increments TokenVersion. Old JWTs fail current-version validation. Android encrypts session; web stores it per tab.
Input / transport	DTO validation + finite-value domain checks. Release Android requires HTTPS; configured CORS origins govern browser access. Login and registration limited to 20 requests/minute per IP.
Errors / offline	Problem Details for business failures; database errors return 503. Offline map/reference cache is labelled. Booking and transfer mutations need the central API.
Audit and expiry	Many operational mutations log actor/action/entity in the same transaction. Not every creation/profile action is audited. Expiry processes up to 500 candidates per pass; it is not an immediate real-time scheduler.

Capacity planning and known design limits

Current behavior versus possible future extensions

Current limit / behavior	Implication	Possible extension - not implemented
One API host, Mongo transactions	Simple deployment; database must be a replica set. Single-member local setup has no host redundancy.	Load-balanced API instances and multi-member Mongo deployment with coordinated operations.
Node/user/slot list limit: 500; reservations paginated	Suitable for current project scale; map is not an unlimited catalogue. View() joins user and station per booking.	Paginate catalogues; batch booking joins; geospatial indexes if scale requires them.
Staff-assisted booking without consent check	Staff can create a Pending booking for an active customer.	Explicit customer confirmation and consent history.
Shared holds for both energy directions	No separate import/export inventory or network constraints. Completion releases reserved energy counters.	Direction-specific supply model and metered cumulative throughput.
Manual actual-energy entry; no prices/payments	QR proves booking authenticity, not a hardware measurement or financial settlement.	Meter/charger integration, pricing, settlement and notifications.

Implementation references

These are repository paths, not external product documentation. The code is the source of truth for this guide.

backend/SolarTrading.Api/Services/AccountService.cs, GridService.cs and ReservationService.cs

backend/SolarTrading.Api/Services/BusinessRules.cs and ReservationExpiryService.cs

backend/SolarTrading.Api/Program.cs and Repositories/MongoStore.cs

frontend/android/app/src/main/java/com/solara/microgrid/MapActivity.java